UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
FACULTAD DE INGENIERÍA
Ingeniería en Ciencias y Sistemas

MiniShell: Intérprete de Comandos en ARM64

Arquitectura de Computadores y Ensambladores 1
Segundo semestre 2026

Estudiante: Luis Ángel Saloj Cuma
Carné: 202503517
Sección: B

1. Introducción

MiniShell es un intérprete de comandos desarrollado en ensamblador ARM64 para Linux. El proyecto permite recibir instrucciones escritas por el usuario, reconocer comandos internos y ejecutar programas externos mediante llamadas al sistema. Su implementación busca demostrar conceptos de arquitectura de computadores, manejo de registros, memoria, pila y comunicación con el sistema operativo.

2. Objetivos

2.1 Objetivo general

Implementar un intérprete de comandos básico en ensamblador ARM64 que permita interactuar con Linux mediante comandos internos y la ejecución de programas externos.

2.2 Objetivos específicos

- Aplicar instrucciones del conjunto de instrucciones AArch64 mediante GNU Assembler (GAS).
- Leer y procesar la entrada del usuario desde la terminal.
- Reconocer comandos internos como help, echo, clear y exit.
- Ejecutar programas externos mediante las llamadas al sistema correspondientes.
- Utilizar QEMU para ejecutar el binario ARM64 y GDB para observar la ejecución.

3. Herramientas y entorno

- GNU Assembler para AArch64 (aarch64-linux-gnu-as): ensamblado del código fuente.
- GNU linker para AArch64 (aarch64-linux-gnu-ld): creación del ejecutable.
- Make: automatización del proceso de compilación.
- QEMU user-mode (qemu-aarch64): ejecución del programa ARM64 en el entorno disponible.
- GDB multiarch: depuración remota e inspección de registros, pila e instrucciones.
- Git y GitHub: control de versiones y publicación del proyecto.

4. Descripción general del funcionamiento

Al iniciar, MiniShell muestra un indicador de entrada y espera una línea escrita por el usuario. El programa procesa esa línea, separa el comando de sus argumentos y determina si corresponde a una función interna. Si es un comando interno, ejecuta la rutina correspondiente. Si no lo es, intenta ejecutar el programa externo indicado por una ruta absoluta. El intérprete continúa aceptando comandos hasta recibir exit.

5. Comandos demostrados

5.1 help

Muestra la ayuda disponible y los comandos internos reconocidos por el intérprete.

5.2 echo

Imprime en la terminal el texto proporcionado como argumento.

5.3 clear

Limpia la pantalla de la terminal, de acuerdo con la implementación del proyecto.

5.4 exit

Finaliza la ejecución de MiniShell.

5.5 Ejecución de programas externos

El proyecto admite la ejecución de programas externos mediante rutas absolutas. Ejemplos probados incluyen:

- `/bin/echo comando externo`
- `/bin/uname -a`
- `/comando_inexistente`

El primer y segundo ejemplo ejecutan programas del sistema. El tercero sirve para observar el comportamiento ante una ruta que no corresponde a un ejecutable disponible.

6. Organización del proyecto

- Makefile: define las reglas para ensamblar y enlazar el programa.
- README.md: contiene instrucciones de uso, compilación y depuración.
- src/main.s: archivo principal de código ensamblador.
- src/main_objetivo3.s: archivo de código adicional conservado en el repositorio.
- evidencias/gdb_sesion.txt: registro de una sesión de depuración.
- .gitignore: excluye el directorio build/ y el ejecutable minishell.

7. Compilación y ejecución

Desde el directorio raíz del proyecto, se puede compilar con:

```
make
```

Para ejecutar el binario ARM64 mediante QEMU:

```
qemu-aarch64 ./minishell
```

El archivo Makefile invoca el ensamblador cruzado para producir el objeto y luego el enlazador para generar el ejecutable. Los archivos generados se excluyen del control de versiones mediante el archivo .gitignore.

8. Registros, pila y llamadas al sistema

En AArch64, los registros de propósito general se identifican como x0–x30. En convenciones habituales de llamadas a funciones, x0–x7 se utilizan para argumentos y valores de retorno, mientras que x30 contiene la dirección de retorno (link register). El registro sp señala la pila y x29 suele emplearse como puntero de marco en funciones que lo necesitan.

Las llamadas al sistema de Linux ARM64 reciben el número de syscall en x8 y sus argumentos en registros x0–x5; la instrucción svc #0 transfiere el control al kernel. En la implementación del proyecto se utilizan, entre otras, las siguientes llamadas:

- read (63): lectura de datos desde la entrada estándar.
- write (64): escritura de texto en la salida estándar.
- exit (93): terminación de un proceso.
- clone (220): creación del proceso hijo usado para ejecutar comandos externos.
- execve (221): reemplazo de la imagen del proceso hijo por el programa solicitado.
- wait4 (260): espera y recolección del estado del proceso hijo.

La ejecución externa usa clone con las opciones configuradas por el programa para crear un hijo; el hijo intenta ejecutar el programa con execve y el proceso principal espera su finalización con wait4.

9. Pruebas funcionales

Las pruebas realizadas durante el desarrollo incluyeron:

- Compilación con make sin errores.
- Inicio del intérprete y visualización del prompt MiniShell>.
- Uso del comando help.
- Impresión de texto mediante echo.
- Ejecución de programas externos mediante rutas absolutas.
- Prueba de una ruta de ejecutable inexistente.
- Finalización del intérprete mediante exit.
- Inspección con GDB de puntos de interrupción, registros, memoria de pila e instrucciones.

10. Depuración con GDB

La depuración se realizó mediante QEMU en modo de espera para conexión remota y GDB multiarch. Se establecieron puntos de interrupción en rutinas como comparar_cadenas, comando_echo y ejecutar_externo. Durante la sesión se inspeccionaron los registros del procesador, la memoria referenciada por sp y las instrucciones próximas al contador de programa pc.

```
Terminal 1:  qemu-aarch64 -g 1234 ./minishell

Terminal 2:  gdb-multiarch ./minishell
```

Comandos de referencia utilizados en GDB:

- `set architecture aarch64`
- `set pagination off`
- `set debuginfod enabled off`
- `target remote :1234`
- `break comparar_cadenas`
- `break comando_echo`
- `break ejecutar_externo`
- `continue`
- `info registers`
- `x/8gx $sp`
- `x/6i $pc`

La evidencia textual de la sesión se conserva en evidencias/gdb_sesion.txt dentro del repositorio.

11. Limitaciones conocidas

- Los comandos externos se invocan mediante rutas absolutas; no se implementa una búsqueda PATH completa.
- MiniShell es una implementación educativa y no reemplaza un shell completo como Bash.
- La compatibilidad de los programas externos depende de que existan en el sistema donde se ejecuta QEMU.
- El comportamiento de señales, redirecciones, tuberías y expansión de variables no forma parte de las funciones descritas aquí.

12. Conclusiones

El desarrollo de MiniShell permitió practicar programación de bajo nivel en ARM64, ensamblado y enlazado, uso de llamadas al sistema de Linux y depuración con GDB. La implementación proporciona una interfaz básica para comandos internos y ejecución de programas externos, y el repositorio incluye el código fuente, las instrucciones de compilación y una evidencia de depuración.

13. Referencias y recursos

- Documentación de GNU Assembler (GAS) y GNU Binutils.
- Documentación de QEMU user-mode.
- Manual de GDB.
- Documentación de llamadas al sistema de Linux para AArch64.
- Repositorio del proyecto: https://github.com/salojmmp/ARQUI1B_2S_P2_202503517